

Turtle ID

Kaplumbaga Fotograf Tanim Sistemi

Clean Code · SOLID · Multi-Agent Mimari · Teknik Rapor

Proje:	HW_Turtle — Kaplumbaga Re-Identification Sistemi
Dil:	Python 3.11+
Mimari:	Multi-Agent + Hexagonal (Port/Adapter)
Arayuz:	PySide6 (Qt 6)
Veritabani:	SQLite + SQLAlchemy 2.0 (WAL modu)
Model:	EfficientNet-B0 (timm) — 1280 boyutlu embedding
Esleme:	Cosine Similarity (scikit-learn)
GitHub:	github.com/YusufAltunbay/HW_Turtle

3.974

satir kaynak kodu

6

ajan

4

port arayuzu

41

test — %100 gecme

21

git commit

ICERIK

- Proje Ozeti ve Mimari Genel Bakis
- SOLID Prensipleri — Somut Kod Ornekleri
- Clean Code Prensipleri
- Multi-Agent Mimarisi
- EventBus Mekanizmasi
- Port / Adapter (Hexagonal) Mimarisi
- Is Akislari (Kayit & Dogrulama)
- Test Yapisi
- Kod Istatistikleri ve Ozet

1. Proje Ozeti ve Mimari Genel Bakis

Turtle ID, farkli zamanlarda ve kosullarda cekilen kaplumbaga fotoğraflarından aynı bireyi taniyan gorsel esleme sistemidir. EfficientNet-B0 sinir agi her fotoğraftan 1280 boyutlu L2-normalize bir ozellik vektörü cikarir; cosine benzerligi ile sorgu vektörü veritabanındaki tüm vektorlerle karsilastirilerek en iyi eslesme bulunur.

Teknoloji Yigini

Katman	Teknoloji	Gorevin Ozeti
Goruntu ozelligi	EfficientNet-B0 (timm)	1280-dim L2-normalize vektor
On isleme	OpenCV	Letterbox + CLAHE + ImageNet normalize
Esleme	Cosine Similarity (sklearn)	Matris carpimiyla toplu karsilastirma
Veritabani	SQLite + SQLAlchemy 2.0	WAL modu, soft delete, BLOB vektor
Kullanici arayuzu	PySide6 (Qt 6)	QThread worker'lar, Signal/Slot
Ajan iletisimi	Thread-safe EventBus	Pub/sub, hata izolasyonu
DI	Manuel AppContainer	Tum bagimliliklar tek noktada

Klasor Yapisi

Proje dizin agaci

```
src/turtle_id/
  agents/          7 ajan (BaseAgent'tan turetilmis)
  core/
    models/        4 domain modeli (Turtle, Embedding, MatchResult)
    ports/          4 soyut arayuz (ITurtleRepository, IMatcher, ...)
    use_cases/      2 is akisi (Register, Verify)
  infrastructure/
    persistence/    SQLite + SQLAlchemy ORM
    vision/          EfficientNet-B0, OpenCV on isleme
    matching/        Cosine Similarity matcher
  ui/
    views/          5 gorsel sayfa
    widgets/         2 ozel widget
  shared/           EventBus + EventType enum (13 olay)
  app.py            Giris noktasi
  container.py      Bagimlilik enjeksiyon konteyneri
```

Katmanli Mimari Diyagram

Mimari katmanlari

UI KATMANI

```
PySide6 MainWindow -> RegisterView / VerifyView
QThread Workers    -> RegisterWorker / VerifyWorker
|
| cagri (use case)
v
```

USE CASE KATMANI

```
RegisterTurtleUseCase    VerifyTurtleUseCase
(frozen dataclass DTO'lar ile tip guvenligi)
|
| ajan cagrısı
v
```

AJAN KATMANI

```
PhotoValidationAgent  ImageAgent  MatchingAgent
DataAgent              ConfigAgent
----- EventBus (pub/sub) -----
|
| port arayuzu (DIP)
v
```

ALTYAPI KATMANI (Adapters)

```
OpenCVPreprocessor      TimmEfficientNetExtractor
CosineMatcher           SQLiteTurtleRepository
```

2. SOLID Prensipleri — Somut Kod Ornekleri

2.1 SRP — Single Responsibility Principle

Her sınıf ve ajan yalnızca bir nedenden değişir. PhotoValidationAgent sadece fotoğraf kalitesini kontrol eder; embedding üretmez, veritabanına yazmaz, esleme yapmaz.

Sınıf / Ajan	Tek Sorumlulugu	Yapmadigi
EventBus	Pub/sub mekanizması	İs mantığı, veri erişimi
ConfigAgent	Ayar okuma / yazma	Esleme, embedding
PhotoValidationAgent	Fotoğraf kalitesi kontrolü	Kayıt, esleme
ImageAgent	Embedding vektörü üretimi	Veritabanı, esleme
MatchingAgent	Cosine esleme	Veri kaydetme, UI
DataAgent	CRUD işlemleri	Görüntü işleme
OpenCVPreprocessor	Fotoğraf ön işleme	Feature extraction
TimmEfficientNetExtractor	Feature extraction	Ön işleme, esleme
CosineMatcher	Benzerlik hesabi	Vektör çıkartma
SQLiteTurtleRepository	SQLite erişimi	İs mantığı
RegisterTurtleUseCase	Kayıt akisi orkestrasyonu	Veri erişimi
VerifyTurtleUseCase	Doğrulama akisi	Kayıt, depolama

Somut Örnek — PhotoValidationAgent

SRP: PhotoValidationAgent — her metod tek iş yapar

```
class PhotoValidationAgent(BaseAgent):
    # 7 küçük, tek amaçlı metod:
    def _check_file_exists(self, path, result):    # ~10 satır
        ...
    def _check_extension(self, path, result):      # ~8 satır
        ...
    def _check_blur(self, img, result):            # Laplacian variance > 50
        ...
    def _check_brightness(self, img, result):      # 20 < ort. < 240
        ...

    # Ana metod sadece orkestre eder — is mantığı YOKTUR:
    def validate(self, image_path) -> ValidationResult:
        result = ValidationResult(is_valid=True)
        self._check_file_exists(path, result)
        self._check_extension(path, result)
        self._check_blur(img, result)
        self._check_brightness(img, result)
        return result
```

2.2 OCP — Open / Closed Principle

Mevcut kod değiştirilmeden genişletilebilir. Port arayüzleri sayesinde yeni bir esleme algoritması veya

veritabani eklemek icin yalnızca container.py'da 1-2 satir degismesi yeterlidir.

OCP: Yeni esleme algoritmasi — tek satirlik degisiklik

```
# Mevcut adapter:
class CosineMatcher(IMatcher):
    def find_best_match(self, query, candidates, threshold, top_k):
        scores = cosine_similarity([query], matrix)[0]
        ...

# Yeni adapter (MEVCUT KOD DEGISMEZ):
class FAISSMatcher(IMatcher):
    def find_best_match(self, query, candidates, threshold, top_k):
        # Milyonlarca embedding icin GPU hizli FAISS
        ...

# container.py'da TEK SATIR:
matcher = FAISSMatcher() # onceki: CosineMatcher()
```

Degistirilebilir Adapter'lar

Port (Soyut)	Mevcut Adapter	Alternatif Adapter
ITurtleRepository	SQLiteTurtleRepository	PostgreSQLRepository
ImagePreprocessor	OpenCVPreprocessor	PytorchPreprocessor
IEmbeddingExtractor	TimmEfficientNetExtractor	ResNet50Extractor
IMatcher	CosineMatcher	FAISSMatcher

2.3 LSP — Liskov Substitution Principle

Turetilmis siniflar temel sinifin yerine gelecek sekilde tasarlanmistir. container.py'da tum ajanlar BaseAgent listesiyle yonetilir.

LSP: Polimorfik ajan yonetimi

```
# container.py – tum ajanlar BaseAgent listesiyle yonetilir:
agents: list[BaseAgent] = [
    self._validation_agent, # PhotoValidationAgent
    self._image_agent,      # ImageAgent
    self._matching_agent,   # MatchingAgent
    self._data_agent,       # DataAgent
]

for agent in agents:
    agent.start()           # Her biri BaseAgent.start() sozlesmesini uygular
    agent.stop()            # Her biri BaseAgent.stop() uygular
    agent.health_check()    # Polimorfik – tip bilmek gerekmez
```

2.4 ISP — Interface Segregation Principle

Her port arayuzu minimal ve ozguldur. Bir sinif kullanmayacagi metotlari implement etmek zorunda kalmaz.

ISP: Minimal port arayuzleri

```
class IImagePreprocessor(ABC):
    @abstractmethod
    def preprocess(self, image_path: str) -> np.ndarray: ...
    # TEK METOD – ImageAgent'in ihtiyaci olan tek sey

class IMatcher(ABC):
    @abstractmethod
    def find_best_match(self, query, candidates,
                        threshold, top_k) -> MatchResult: ...
    # TEK METOD – MatchingAgent baska bir sey bilmez
```

2.5 DIP — Dependency Inversion Principle

Yuksek seviyeli moduller dusuk seviyeli modullere bagimli degil; ikisi de soyutlamalara (port arayuzlerine) bagimlidir. Tum somut baglantilar AppContainer'da tek noktada toplanir.

DIP: Constructor injection ornegi

```
# KOTU – tight coupling (somut sinifa bagimlilik):
class ImageAgent:
    def __init__(self):
        self.preprocessor = OpenCVPreprocessor()          # concrete!
        self.extractor    = TimmEfficientNetExtractor()    # concrete!

# IYI – DIP (soyut arayuze bagimlilik):
class ImageAgent(BaseAgent):
    def __init__(self,
                    event_bus: EventBus,
                    preprocessor: IImagePreprocessor,      # soyut port
                    extractor: IEmbeddingExtractor):        # soyut port
        self._preprocessor = preprocessor
        self._extractor    = extractor

# AppContainer – TEK BIRLESIM NOKTASI:
preprocessor = OpenCVPreprocessor()          # concrete burada
extractor    = TimmEfficientNetExtractor()    # concrete burada
image_agent  = ImageAgent(event_bus,
                          preprocessor,        # enjekte edilir
                          extractor)          # enjekte edilir
```

DIP'in Pratikte Yansimasi

Unit testlerde OpenCV veya PyTorch yuklenmeden mock IImagePreprocessor enjekte edilebilir. Gercek model yerine sahte 1280-dim vektor donduren bir nesne tum test suiti hizli calistirir (41 test ~ 4 saniye).

3. Clean Code Prensipleri

3.1 Anlamli Isimlendirme

Her sinif, metod ve degisken ismi ne yaptigini ya da ne oldugunu dogrudan anlatir. Kisaltmalar ve muphemlikler kullanilmaz.

Anlamli isimlendirme ornekleri

```
# Sinif isimleri – ne yaptiklarini anlatir:
PhotoValidationAgent      # Fotograf dogrular
TimmEfficientNetExtractor # timm ile EfficientNet embedding cikarir
SQLiteTurtleRepository    # SQLite ile kaplumbaga verileri
RegisterTurtleUseCase     # Kaplumbagayi kaydeden is akisi
CosineMatcher             # Cosine benzerligi ile eslestirir

# Metod isimleri – fiil + nesne:
validate(image_path)      # ne yaptiyor: validate
find_best_match(query)    # ne yaptiyor: find + best match
add_embedding(embedding)  # ne yaptiyor: add
get_all_embeddings()      # ne dondurdugu belli
has_embeddings()          # bool – soru sorar
is_normalized()           # bool – soru sorar
health_check()            # ne kontrol ettigi belli

# Degisken isimleri:
similarity_score = 0.876  # magic number degil, isimlendirilmis
matched_turtle_id        # ne oldugu belli
top_candidates            # list, ne icerdigi belli
```

3.2 Kucuk Fonksiyonlar (Tek Is)

Her metod tek bir isi yapar. PhotoValidationAgent'in validate() metodu sadece orkestre eder; asil is 7 kucuk private metoda dagitilmistir.

Kucuk fonksiyonlar: validate() sadece orkestre eder

```
def validate(self, image_path: str) -> ValidationResult:
    result = ValidationResult(is_valid=True)
    path = Path(image_path)

    self._check_file_exists(path, result)    # 10 satir
    if not result.is_valid:
        return result                        # erken cikis – deep nesting yok

    self._check_extension(path, result)      # 8 satir
    self._check_image_readable(path, result) # 12 satir
    self._check_dimensions(img, result)      # 10 satir
    self._check_blur(img, result)            # 12 satir
    self._check_brightness(img, result)      # 12 satir

    return result # Orkestrasyonun tek gorevi
```

3.3 Hata Yonetimi

Hata yonetimi stratejileri (3 farkli katman)


```
# 1. EventBus – hata IZOLASYONU:
for handler in handlers:
    try:
        handler(event_type, payload)
    except Exception as exc:
        logger.error(f"Handler hatasi: {exc}")
        # Diger handler'lar calismaya devam eder

# 2. Use Case – hatalar EXCEPTION degil DTO ile tasinir:
validation = self._validation_agent.validate(request.photo_path)
if not validation.is_valid:
    return RegisterTurtleResponse(
        success=False,
        errors=validation.errors,    # UI'a iletilir
        warnings=validation.warnings,
    )

# 3. Repository – transaction GUVENLIGI:
with self._session_factory() as session:
    session.add(orm_object)
    session.commit()
# Context manager hata durumunda otomatik rollback yapar
```

3.4 Type Hints ve Frozen DTO

Type hints ve frozen DTO ornekleri

```
from __future__ import annotations # PEP 563 – lazy evaluation

# Tum metodlarda tam tip notasyonu:
def match(self, query_vector: np.ndarray) -> MatchResult: ...
def get_turtle(self, turtle_id: UUID) -> Turtle | None: ...
def list_turtles(self) -> list[Turtle]: ...

# Frozen dataclass – immutable DTO (yanlislikla mutasyon engellenir):
@dataclass(frozen=True)
class RegisterTurtleRequest:
    name: str
    photo_path: str
    species: str = ""
    notes: str = ""

@dataclass(frozen=True)
class VerifyTurtleRequest:
    photo_path: str
```

3.5 DRY — Don't Repeat Yourself

DRY: mapper ve kisa yol ornekleri

```
# Mapper metodu tek yerde tanimli, her yerde kullanilir:
@staticmethod
def _embedding_from_orm(row: EmbeddingORM) -> Embedding:
    vector = np.frombuffer(row.vector_blob, dtype=np.float32).copy()
    return Embedding(turtle_id=UUID(row.turtle_id), vector=vector, ...)

@staticmethod
def _turtle_from_orm(row: TurtleORM) -> Turtle:
    turtle = Turtle(id=UUID(row.id), name=row.name, ...)
    for emb_row in row.embeddings:
        turtle.add_embedding(
            SQLiteTurtleRepository._embedding_from_orm(emb_row) # tekrar kullanim
        )
    return turtle

# BaseAgent kisa yollar – DRY:
def _publish(self, event_type, payload=None):
    self._event_bus.publish(event_type, payload) # tek satirda sarmalanmis
```

4. Multi-Agent Mimarisi

Sistem 6 ozek ajandan olusur. Her ajan BaseAgent'tan turetilir, kendi yasam dongusuyle yonetilir ve EventBus uzerinden haberlesir. Ajanlar birbirini dogrudan import etmez.

4.1 Ajan Tanim Tablosu

Ajan	Sorumluluk	Dinledigi	Yayimladigi
ConfigAgent	settings.json yonetimi	—	CONFIG_CHANGED
PhotoValidationAgent	Fotograf kalite kontrolu	—	PHOTO_VALID
ImageAgent	Embedding vektoru uretimi	—	PHOTO_DOWNLOAD_READY
MatchingAgent	Cosine esleme	CONFIG_CHANGED	MATCH_FOUND
DataAgent	CRUD + dogrulama logu	MATCH_FOUND	MATCH_NOT_FOUND
UIAgent	PySide6 sayfa yonetimi	—	—

4.2 BaseAgent Sozlesmesi

BaseAgent: Template Method Pattern ile temel sozlesme

```
class AgentStatus(Enum):
    IDLE = auto()
    BUSY = auto()
    ERROR = auto()
    STOPPED = auto()

class BaseAgent(ABC):
    @property
    @abstractmethod
    def name(self) -> str: ...          # Her ajanin benzersiz adi

    def start(self) -> None:             # Baslat + handler kaydet
        self._register_handlers()

    def stop(self) -> None: ...          # Durdur + kaynaklar serbest

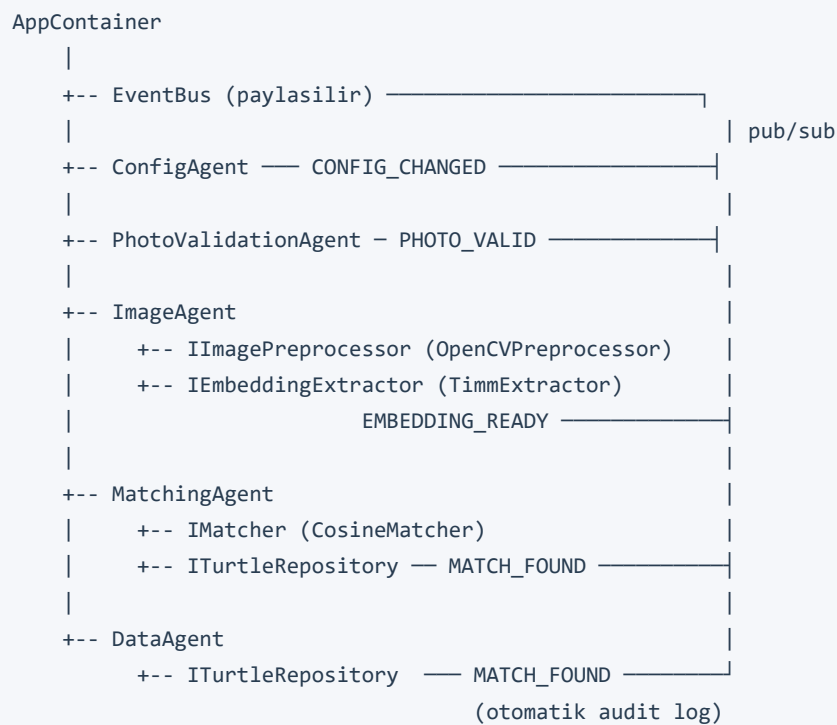
    def health_check(self) -> bool:      # ERROR/STOPPED ise False
        return self._status not in (AgentStatus.ERROR, AgentStatus.STOPPED)

    def _register_handlers(self) -> None:
        # Alt siniflar override eder – Template Method Pattern
        pass

    def _publish(self, event_type, payload=None):
        self._event_bus.publish(event_type, payload) # kisa yol

    def _subscribe(self, event_type, handler):
        self._event_bus.subscribe(event_type, handler)
```

4.3 Ajan Bagimliliklari



5. EventBus Mekanizması

EventBus, ajanlar arası doğrudan import'u ortadan kaldıran merkezi iletişim katmanıdır. Thread-safe pub/sub modeli uygular; bir handler'in hatası diğer handler'ları etkilemez.

5.1 Thread-Safe Tasarım

EventBus: Thread-safe implementasyon ve hata izolasyonu

```
class EventBus:
    def __init__(self):
        self._handlers = defaultdict(list)
        self._lock = threading.Lock()

    def subscribe(self, event_type, handler):
        with self._lock:
            # Kritik bölge (KISA)
            self._handlers[event_type].append(handler)

    def publish(self, event_type, payload=None):
        with self._lock:
            handlers = list(self._handlers.get(event_type, []))
            # LOCK BURADA BITİYOR – handler çağırısı lock dışında
            # Neden? Handler baska bir olay publish ederse DEADLOCK olmaz

            for handler in handlers:
                try:
                    handler(event_type, payload)
                except Exception as exc:
                    logger.error(f"Handler hatası: {exc}")
                    # Diğer handler'lar çalışmaya devam eder
```

5.2 Olay Tipleri (13 EventType)

Kategori	Olay Tipleri	Yayımlayan
Fotoğraf	PHOTO_VALIDATION_STARTED	PhotoValidationAgent
Embedding	EMBEDDING_STARTED	ImageAgent
Esleme	EMBEDDING_STARTED / EMBEDDING_FAILED	MatchingAgent
Veri	TURTLE_SAVED / TURTLE_LOADED	DataAgent
Konfigurasyon	CONFIRMATION_CHANGED	ConfigAgent

5.3 Tight Coupling vs EventBus

EventBus: Loose coupling vs tight coupling karşılaştırması

```
# KOTU – Tight Coupling:
class DataAgent:
    def __init__(self, matching_agent: MatchingAgent):
        # DataAgent, MatchingAgent'i import etmek zorunda
        # -> Circular import riski
        # -> Test ederken her ikisini de hazırlamak gerekir
        self._matching = matching_agent

# IYI – EventBus ile Loose Coupling:
class MatchingAgent(BaseAgent):
    def match(self, query_vector):
        result = self._matcher.find_best_match(...)
        self._publish(EventType.MATCH_FOUND, result) # Kime gideceğini bilmez

class DataAgent(BaseAgent):
    def _register_handlers(self):
        self._subscribe(EventType.MATCH_FOUND, self._on_match_found)

    def _on_match_found(self, event_type, result):
        self._save_verification_log(result) # Otomatik audit log
```

6. Port / Adapter (Hexagonal) Mimarisi

Hexagonal mimaride iç katmanlar (domain, use cases) dışarıdaki sistemleri (SQLite, OpenCV, timm) doğrudan tanımaz. Port arayüzleri bu iki dünyayı birbirinden ayırır.

Hexagonal mimaride katman bağımlılık kuralı

```

UI KATMANI (PySide6)
    |
    v  çağır
USE CASE KATMANI (RegisterTurtle, VerifyTurtle)
    |
    v  port arayuzu (DIP)
CORE / DOMAIN (Turtle, Embedding, MatchResult)
    ^
    | implement eder
ADAPTER KATMANI
    |
    v  kullanır
HARICI SİSTEMLER (SQLite, OpenCV, timm, sklearn)

```

6.1 Port Eslesme Tablosu

Port (ABC)	Adapter	Harici Sistem
ITurtleRepository	SQLiteTurtleRepository	SQLAlchemy + SQLite
IImagePreprocessor	OpenCVPreprocessor	OpenCV + numpy
IEmbeddingExtractor	TimmEfficientNetExtractor	timmm + PyTorch
IMatcher	CosineMatcher	scikit-learn

6.2 Adapter Degistirme Ornegi

OCP + DIP: Adapter degistirmek 1-2 satir

```

# Senaryo: SQLite yerine PostgreSQL kullanmak

# Adım 1 – Yeni adapter (ITurtleRepository implement et):
class PostgreSQLRepository(ITurtleRepository):
    def __init__(self, connection_string: str): ...
    def save(self, turtle: Turtle) -> Turtle: ...
    def find_by_id(self, turtle_id: UUID) -> Turtle | None: ...
    # ... diğer metodlar

# Adım 2 – container.py'da TEK SATIRLIK degisiklik:
# repository = SQLiteTurtleRepository(session_factory)
repository = PostgreSQLRepository("postgresql://user:pass@host/db")

# Hiçbir use case, ajan veya test degismez.

```

6.3 Repository Pattern

Repository pattern: Domain <-> ORM donusumu

```
# Domain katmani SQLite detayini hic gormez:
class RegisterTurtleUseCase:
    def execute(self, request):
        turtle = Turtle(name=request.name, ...)
        # Veritabani mi? Redis mi? PostgreSQL mi? Bilmez.
        saved = self._data_agent.save_turtle(turtle)
        return RegisterTurtleResponse(success=True, turtle=saved)

# Vektor BLOB serializasyonu yalnızca repository'de:
@staticmethod
def _embedding_to_orm(emb: Embedding) -> EmbeddingORM:
    return EmbeddingORM(
        vector_blob = emb.vector.astype(np.float32).tobytes(), # BLOB
        vector_dim = emb.dimension,
        ...
    )

@staticmethod
def _embedding_from_orm(row: EmbeddingORM) -> Embedding:
    vector = np.frombuffer(row.vector_blob, dtype=np.float32).copy()
    return Embedding(vector=vector, ...) # Domain nesnesine donusturuluyor
```


7. Is Akislari — Kayit ve Dogrulama

7.1 Kaplumbaga Kayit Akisi

RegisterTurtleUseCase akisi (adim adim)

```
Kullanici 'Kaydet' tiklar
|
v
RegisterWorker(QThread).run()      <- UI thread'i bloke etmez
|
v
RegisterTurtleUseCase.execute(RegisterTurtleRequest)
|
+--> 1. PhotoValidationAgent.validate(photo_path)
|     +-- PHOTO_VALIDATION_STARTED yayimla
|     +-- _check_file_exists(), _check_extension()
|     +-- _check_dimensions(), _check_blur(), _check_brightness()
|     +-- PHOTO_VALID yayimla (ya da PHOTO_INVALID -> erken donus)
|
+--> 2. Turtle nesnesi olustur (UUID otomatik, kayit tarihi otomatik)
|
+--> 3. ImageAgent.process(photo_path, turtle.id)
|     +-- EMBEDDING_STARTED yayimla
|     +-- OpenCVPreprocessor: letterbox + CLAHE + ImageNet normalize
|     +-- TimmEfficientNetExtractor: EfficientNet-B0 -> (1280,)
|     +-- L2 normalize -> norm ~= 1.0
|     +-- EMBEDDING_READY yayimla -> Embedding nesnesi
|
+--> 4. turtle.add_embedding(embedding)
|
+--> 5. DataAgent.save_turtle(turtle)
|     +-- SQLiteTurtleRepository.save()
|     |   +-- TurtleORM olustur
|     |   +-- EmbeddingORM olustur (vector -> float32 bytes BLOB)
|     |   +-- session.commit()
|     +-- TURTLE_SAVED yayimla

Qt Signal -> Ana thread -> UI: 'Alpha basariyla kaydedildi'
```

7.2 Kaplumbaga Dogrulama Akisi

VerifyTurtleUseCase akisi (adim adim)

```

Kullanici 'Dogrula' tiklar
|
v
VerifyWorker(QThread).run()
|
v
VerifyTurtleUseCase.execute(VerifyTurtleRequest)
|
+--> 1. PhotoValidationAgent.validate(photo_path)
|      [ayni kontroller – PHOTO_VALID veya erken donus]
|
+--> 2. ImageAgent.process(photo_path, uuid4()) <- gecici UUID
|      [ayni pipeline – EMBEDDING_READY]
|
+--> 3. MatchingAgent.match(query_embedding.vector)
|      +-- MATCHING_STARTED yayimla
|      +-- Repository.get_all_embeddings() -> N vektor
|      +-- np.stack() -> (N, 1280) matris
|      +-- cosine_similarity(query, matris) -> (N,) skorlar
|      +-- argsort descending -> top-3 aday
|      +-- best_score >= threshold (varsayilan 0.82)?
|          Evet -> MATCH_FOUND yayimla ----+
|          Hayir -> MATCH_NOT_FOUND ----+
|                                     |
|                                     DataAgent dinler <-----+
|                                     VerificationLogORM: audit trail otomatik
|
+--> 4. Eslesme varsa DataAgent.get_turtle(matched_id)
|      +-- TURTLE_LOADED yayimla
|
+--> 5. match_result.matched_turtle = turtle

Qt Signal -> Ana thread -> ResultView: eslesme rozeti + guven skoru

```

7.3 Audit Trail — Otomatik Dogrulama Logu

Her dogrulama islemi VerificationLogORM tablosuna otomatik yazilir. DataAgent, MATCH_FOUND ve MATCH_NOT_FOUND olaylarini dinler; use case bu logu hic bilmez — tamamen EventBus uzerinden saglanir.

Otomatik audit trail — EventBus ile SRP korunur

```
class DataAgent(BaseAgent):
    def _register_handlers(self):
        # Her dogrulamayi otomatik logla – use case bunu bilmez:
        self._subscribe(EventType.MATCH_FOUND, self._on_match_found)
        self._subscribe(EventType.MATCH_NOT_FOUND, self._on_match_not_found)

    def _on_match_found(self, _event_type, result: MatchResult):
        self._save_verification_log(result)

    def _save_verification_log(self, result):
        log = VerificationLogORM(
            id = str(uuid4()),
            matched_turtle_id= str(result.matched_turtle_id),
            similarity_score = result.similarity_score,
            confidence_score = result.confidence,
            threshold_used = result.threshold_used,
            is_match = result.is_match,
            verified_at = datetime.now(timezone.utc),
        )
        session.add(log)
        session.commit()
```

8. Test Yapisi

8.1 Test Dosyalari

Dosya	Tip	Test Sayisi	Kapsam
test_event_bus.py	Birim	7	Pub/sub, thread safety (20 thread), hata izolasyonu
test_matcher.py	Birim	7	Cosine esleme, esik mantigi, top-k, edge cases
test_models.py	Birim	13	Turtle, Embedding, MatchResult, confidence hesabi
test_validation_agent.py	Birim	4	Fotograf dogrulama + EventBus entegrasyonu
test_repository.py	Entegrasyon	10	SQLite CRUD, vektor saklama, soft delete
TOPLAM		41	%100 gecme oranı

8.2 Ornek Birim Test

test_matcher.py ornekleri

```
class TestCosineMatcher:
    def test_ayni_vektor_eslesir(self, matcher, make_embedding):
        vec = np.random.randn(128).astype(np.float32)
        vec /= np.linalg.norm(vec)          # L2 normalize
        query = vec.copy()
        candidate = make_embedding(vec)

        result = matcher.find_best_match(
            query, [candidate], threshold=0.9, top_k=1
        )

        assert result.is_match is True
        assert result.similarity_score == pytest.approx(1.0, abs=1e-5)

    def test_bos_aday_listesi(self, matcher):
        query = np.ones(128) / np.sqrt(128)
        result = matcher.find_best_match(query, [], threshold=0.75, top_k=3)
        assert result.is_match is False    # Edge case: bos liste
```

8.3 Entegrasyon Test Duzeni

test_repository.py: Vektor BLOB duyarlilik testi

```

@pytest.fixture
def repo(tmp_path):
    """In-memory SQLite – disk IO yok, hizli (< 4 sn toplam)."""
    engine = build_engine(":memory:")
    factory = build_session_factory(engine)
    init_db(engine)
    return SQLiteTurtleRepository(factory)

def test_embedding_vektoru_bozulmadan_geri_gelir(repo, make_turtle):
    """float32 BLOB saklama / geri yukleme duyarlilik testi."""
    original = np.random.randn(1280).astype(np.float32)
    original /= np.linalg.norm(original)

    turtle = make_turtle()
    embedding = Embedding(turtle_id=turtle.id, vector=original, ...)
    turtle.add_embedding(embedding)
    saved = repo.save(turtle)

    loaded = repo.find_by_id(saved.id)
    restored = loaded.embeddings[0].vector

    np.testing.assert_array_almost_equal(original, restored, decimal=5)
    # 5 ondalik basamak hassasiyet – float32 BLOB'da kayip yok

```

8.4 Thread-Safety Testi

test_event_bus.py: 20 thread ile thread-safety dogrulaması

```

def test_thread_safe_publish(self, bus):
    """20 thread ayni anda publish ediyor – kayip veya deadlock olmamali."""
    results = []
    def handler(event_type, payload):
        results.append(payload)

    bus.subscribe(EventType.TURTLE_SAVED, handler)

    threads = [
        threading.Thread(
            target=bus.publish,
            args=(EventType.TURTLE_SAVED, i)
        )
        for i in range(20)
    ]
    for t in threads: t.start()
    for t in threads: t.join()

    assert len(results) == 20 # Hic kayip yok

```

9. Kod Istatistikleri ve Ozet

9.1 Kod Istatistikleri

Metrik	Deger
Kaynak kodu (src/)	3.974 satir
Test kodu (tests/)	493 satir
Toplam Python dosyasi	52 dosya
Ajan sayisi	6 ajan (BaseAgent haric)
Port arayuzu (ABC)	4 arayuz
Adapter implementasyonu	4 adapter
Use case	2 (Register + Verify)
ORM tablosu	3 (turtles, embeddings, verification_log)
UI sayfasi	5 sayfa
Birim testi	31 test
Entegrasyon testi	10 test
Test basari orani	%100 (41/41)
Test suresi	~4 saniye (in-memory SQLite)
Git commit	21 commit

9.2 Uygulanan Prensipler Ozeti

Prensip	Uygulama	Kanit
SRP	12 sinif/ajan tek sorumluluk	PhotoValidationAgent sadece dogrular
OCP	Port arayuzleri	Yeni adapter = 1-2 satir container
LSP	BaseAgent hiyerarsisi	list[BaseAgent] polimorfik yonetim
ISP	Minimal port arayuzleri	ImagePreprocessor: 1 metod
DIP	Constructor injection	Tum ajanlar port arayuzune bagimli
Anlamli isim	Fiil+nesne metod isimleri	find_best_match, has_embeddings
Kucuk fonksiyon	Tek is yapan metodlar	7 private _check_*() metodu
Hata yonetimi	DTO + hata izolasyonu	EventBus handler hatalari izole
DRY	Mapper + BaseAgent kisayollar	_embedding_from_orm tekrar kullanim
Type hints	Tum metodlar notasyonlu	Python 3.11+ frozen dataclass DTO
Pub/Sub	Thread-safe EventBus	Ajanlar birbirini import etmiyor
Hexagonal	Port/Adapter ayirimi	Harici sistem swap edilebilir

9.3 Mimari Guclu Yonler

>>Degistirilebilirlik

Veritabani, model veya esleme algoritmasi degistirmek icin
yalnizca container.py'da 1-2 satir degisiklik yeterlidir.

>>Test edilebilirlik

Port arayuzleri sayesinde mock enjeksiyonu kolaydir.
Entegrasyon testleri in-memory SQLite ile disk IO olmadan calisir.

>>Genisletilebilirlik

Yeni ajan eklemek: BaseAgent'i turet, _register_handlers()
override et, container.py'a ekle.

>>Hata yalitimi

Bir EventBus handler hatasi sistemi durdurmaz.
Use case hatalari exception degil DTO ile tasinir.

>>Dusuk baglilik

Ajanlar birbirini import etmiyor; yalnızca EventBus
ve port arayuzleri üzerinden haberlesiyorlar.

>>Thread safety

EventBus Lock granularitesi optimize edilmistir.
Uzun islemler QThread worker'larinda calisir.

Kaynak: github.com/YusufAltunbay/HW_Turtle